PlayBuf : MultiOutUGen : UGen : AbstractFunction : Object

Sample playback oscillator.

Source: BufIO.sc

See also: RecordBuf, DiskIn, BufRd

Description

Plays back a sample resident in memory.

Class Methods

```
PlayBuf.ar(numChannels, bufnum: 0, rate: 1.0, trigger: 1.0, startPos: 0.0,
      loop: 0.0, doneAction: 0)

PlayBuf.kr(numChannels, bufnum: 0, rate: 1.0, trigger: 1.0, startPos: 0.0,
      loop: 0.0, doneAction: 0)
```

Arguments:

numChannels	Number of channels that the buffer will be. This must be a fixed integer. The architecture of the SynthDef cannot change after it is compiled.
bufnum	The index of the buffer to use. **NOTE:** If you supply a bufnum of a buffer with a differing number of channels than the one specified in this PlayBuf, it will post a warning and output the channels it can.
rate	1.0 is the server's sample rate, 2.0 is one octave up, 0.5 is one octave down -1.0 is backwards normal rate… etc. Interpolation is cubic.
trigger	A trigger causes a jump to the startPos. A trigger occurs when a signal changes from negative value to positive value.
startPos	Sample frame to start playback.
loop	1 means true, 0 means false. This is modulateable.
doneAction	an integer representing an action to be executed when the buffer is finished playing. This can be used to free the enclosing synth, etc. See Done for more detail. `doneAction` is only evaluated if loop is 0.

Inherited class methods

1 methods from MultiOutUGen ► show

7 methods from UGen ► show

8 methods from Object ► show

Instance Methods

Inherited instance methods

5 methods from MultiOutUGen ► show

97 methods from UGen ► show

161 methods from AbstractFunction ► show

Examples

```
s.boot // Boot the server, if you need to

// read a whole sound into memory

// ExampleFiles helps locate audio files used in examples
p = ExampleFiles.child;
// peek at the path to see location and the format of a path
p.postln;
// read the file into a buffer
b = Buffer.read(s, p); // remember to free the buffer later.

SynthDef(\help_PlayBuf, { |out = 0, bufnum = 0|
    Out.ar(out,
        PlayBuf.ar(1, bufnum, BufRateScale.kr(bufnum), doneAction: Done.freeSelf)
    )
}).play(s, [\out, 0, \bufnum, b]);
```

In the above example, note how the `doneAction: Done.freeSelf` causes the synth to free itself when the buffer reaches its end.

Note again that the number of channels must be fixed for the SynthDef. It cannot vary depending on which buffer you use.

```
// loop is true
SynthDef(\help_PlayBuf, { |out = 0, bufnum = 0|
    Out.ar(out,
        PlayBuf.ar(1, bufnum, BufRateScale.kr(bufnum), loop: 1.0)
    )
}).play(s, [\out, 0, \bufnum, b]);


// trigger one shot on each pulse
SynthDef(\help_PlayBuf, { |out = 0, bufnum = 0|
    var trig;
    trig = Impulse.kr(2.0);
    Out.ar(out,
        PlayBuf.ar(1, bufnum, BufRateScale.kr(bufnum), trig, 0, 0)
    )
}).play(s, [\out, 0, \bufnum, b]);


// trigger one shot on each pulse
SynthDef(\help_PlayBuf, { |out = 0, bufnum = 0|
    var trig;
    trig = Impulse.kr(XLine.kr(0.1, 100, 30));
    Out.ar(out,
        PlayBuf.ar(1, bufnum, BufRateScale.kr(bufnum), trig, 5000, 0)
    )
}).play(s, [\out, 0, \bufnum, b]);


// mouse control of trigger rate and startpos
SynthDef(\help_PlayBuf, { |out = 0, bufnum = 0|
    var trig;
    trig = Impulse.kr(MouseY.kr(0.5, 200, 1));
    Out.ar(out,
        PlayBuf.ar(1, bufnum, BufRateScale.kr(bufnum), trig, MouseX.kr(0,
BufFrames.kr(bufnum)), 1)
    )
}).play(s, [\out, 0, \bufnum, b]);


// accelerating pitch
SynthDef(\help_PlayBuf, { |out = 0, bufnum = 0|
    var rate;
```

```
        PlayBuf.ar(1, bufnum, BufRateScale.kr(bufnum)*rate, 1.0, 0.0, 1.0)
    )
}).play(s, [\out, 0, \bufnum, b]);


// sine wave control of playback rate. negative rate plays backwards
SynthDef(\help_PlayBuf, { |out = 0, bufnum = 0|
    var rate;
    rate = FSinOsc.kr(XLine.kr(0.2, 8, 30), 0, 3, 0.6);
    Out.ar(out,
        PlayBuf.ar(1, bufnum, BufRateScale.kr(bufnum) * rate, 1, 0, 1)
    )
}).play(s, [\out, 0, \bufnum, b]);


// zig zag around sound
SynthDef(\help_PlayBuf, { |out = 0, bufnum = 0|
    var rate;
    rate = LFNoise2.kr(XLine.kr(1, 20, 60), 2);
    Out.ar(out,
        PlayBuf.ar(1, bufnum, BufRateScale.kr(bufnum) * rate, 1, 0, 1)
    )
}).play(s, [\out, 0, \bufnum, b]);

b.free;
```

helpfile source: /usr/local/share/SuperCollider/HelpSource/Classes/PlayBuf.schelp
link::Classes/PlayBuf::